

Praktikumsbericht

Canny Edge Detection auf der GPU

Matthias Stefan

Thomas Jurczyk

23. August 2026

Inhaltsverzeichnis

1	Voraussetzungen und Projektaufbau	3
1.1	Einleitung	3
1.2	Hardware-Spezifikationen	3
1.3	Projektstruktur	4
1.4	Build-System und Kompilierung	4
1.5	Messmethodologie	5
1.6	Verwandte Arbeiten	5
2	Algorithmus	7
2.1	Überblick	7
2.2	Schritte	7
2.2.1	Schritt 1: Gaussian Blur	7
2.2.2	Schritt 2: Sobel	7
2.2.3	Schritt 3: Non-Maximum Suppression	8
2.2.4	Schritt 4: Hysteresis Thresholding	9
3	Implementierung	10
3.1	Schritt 1: Gaussian Blur	10
3.1.1	Naive Implementierung	10
3.1.2	Shared Memory Optimierung	10
3.1.3	Pinned Memory Optimierung	10
3.1.4	Warp-Level Optimierung	11
3.2	Schritt 2: Sobel	11
3.2.1	Naive Implementierung	11
3.2.2	Shared-Memory-Variante	12
3.2.3	Arithmetische Optimierung	12
3.2.4	Richtungs-Buckets	12
3.2.5	Split-Variante	12
3.3	Schritt 3: Non-Maximum Suppression	13
3.3.1	Naive Implementierung	13
3.3.2	Shared Memory Optimierung	14
3.3.3	Bucket-Optimierung	14
3.4	Schritt 4: Hysteresis	14

3.4.1	Naive Implementierung	15
3.4.2	Shared-Memory-Optimierung	15
3.4.3	Optimierte Variante und Randbehandlung	15
4	Ergebnisse und Analyse	16
4.1	Gaussian Blur	16
4.1.1	Korrektheit	16
4.1.2	Performance	16
4.2	Schritt 2: Sobel	17
4.2.1	Performance	17
4.2.2	Kernelprofiling	18
4.3	Schritt 3: Non-Maximum Suppression	19
4.3.1	Korrektheit	19
4.3.2	Performance	20
4.4	Schritt 4: Hysteresis	21
4.4.1	Performance	21
4.4.2	Kernelprofiling	22
4.5	Fused Pipeline	22
4.5.1	Konzept	22
4.5.2	Performance	23
5	Fazit	24
5.1	Fazit	24
	Anhang	26
	A Performance-Rohdaten Gaussian Blur	26
	B Sobel Messungen	27
	C NMS Messungen	29
	D Hysteresis Messungen	31

1 Voraussetzungen und Projektaufbau

1.1 Einleitung

Dieser Bericht dokumentiert die im Rahmen des CUDA-Praktikums entstandene Implementierung der Canny Edge Detection auf der GPU. Ziel war es, die einzelnen Stufen des Algorithmus, namentlich Gaussian Blur, Sobel-Filter, Non-Maximum Suppression und Hysteresis Thresholding, als eigene CUDA-Kernel umzusetzen und in mehreren Optimierungsstufen (naiv, Shared Memory, Pinned Memory, sowie stufenspezifische weitere Optimierungen) gegeneinander zu messen und zu analysieren.

Der Bericht gliedert sich in fünf Abschnitte: Zunächst werden die Rahmenbedingungen des Projekts sowie die verwendete Messmethodik beschrieben (Abschnitt 1), gefolgt von einer theoretischen Einordnung des Canny-Algorithmus (Abschnitt 2). Anschließend werden die konkreten CUDA-Implementierungen der vier Pipeline-Schritte vorgestellt (Abschnitt 3) und deren Performance anhand von Laufzeitmessungen sowie NCU-Profilung ausgewertet (Abschnitt 4). Den Abschluss bildet ein gemeinsames Fazit (Abschnitt 5).

1.2 Hardware-Spezifikationen

Die Implementierung wurde auf zwei unterschiedlichen Systemen entwickelt und gemessen.¹ Die Hardware-Ausstattung unterscheidet sich dabei deutlich, was den späteren Vergleich der Ergebnisse interessant macht.

System: Matthias Stefan

Parameter	Wert
GPU	NVIDIA GeForce RTX 2060 (Laptop)
Compute Capability	7.5
Gesamter GPU-Speicher	5737 MB
L2-Cache	3072 KB
Streaming Multiprocessors (SM)	30
Max. Threads pro SM	1024
Warpgröße	32
Memory-Takt	5501 MHz
Memory-Busbreite	192 bit
Peak-Speicherbandbreite	264 GB/s
GPU-Takt	1350 MHz

Tabelle 1: GPU-Spezifikationen der RTX 2060 (Matthias Stefan)

¹Ein Transparenzhinweis zu Nutzung von KI: In dieser Arbeit und während der Messungen wurde KI sowohl für die Korrektur des Textes als auch zur Diskussion der Messerergebnisse verwendet. Auch wurde KI für die Überarbeitung des Codes und partiell auch für dessen Erstellung verwendet. Ein gutes Beispiel ist hier das Schleifen-Unrolling, das als Teil der Optimierung verwendet wurde. Diese sehr mühsame Aufgabe wurde der KI überlassen, der Code aber anschließend überprüft.

System: Thomas Jurczyk

Parameter	Wert
GPU	NVIDIA GeForce RTX 5070 Ti
Compute Capability	12.0
Gesamter GPU-Speicher	16 303 MB
L2-Cache	49 152 KB
Streaming Multiprocessors (SM)	70
Max. Threads pro SM	1536
Warpgröße	32
Memory-Takt	14 001 MHz
Memory-Busbreite	256 bit
Peak-Speicherbandbreite	896 GB/s
GPU-Takt	2588 MHz

Tabelle 2: GPU-Spezifikationen der RTX 5070 Ti (Thomas Jurczyk)

1.3 Projektstruktur

Der Quellcode ist modular nach Pipeline-Stufen organisiert, wobei jede Stufe aus einem `.cu/.cuh`-Paar besteht, das die jeweiligen `execute()`- und `cleanup()`-Funktionen sowie die zugehörigen CUDA-Kernel kapselt. Zum Einlesen und Schreiben der Bilddaten wird die Header-only-Bibliothek `stb_image` eingesetzt.

```
/
├── run_debug.sh
├── run_release.sh
└── src/
    ├── common.cuh
    ├── gaussian.cu(h)
    ├── main.cu
    ├── nms.cu(h)
    ├── sobel.cu(h)
    ├── hysteresis.cu(h)
    └── stb_image(_write).h
```

1.4 Build-System und Kompilierung

Die Optimierungsstufe je Pipeline-Stufe wird zur Kompilierzeit über Preprocessor-Makros gewählt, wodurch für jede Konfiguration ein eigenes, spezialisiertes Binary ohne Laufzeit-Overhead entsteht. Die Stufen bauen dabei aufeinander auf, statt unabhängige Alternativen darzustellen: Bei Gaussian ersetzt Shared Memory zunächst die naive Faltung durch Tiling, während Pinned Memory zusätzlich den D→H-Transfer beschleunigt, sodass die höchste Stufe stets beides kombiniert. Selbiges gilt für den Sobel Filter, bei dem die Pinned Memory Variante den optimierten Kernel enthält.

`build_release.sh` übersetzt jede Übersetzungseinheit einzeln mit `nvcc` und reicht die Stufe über die Umgebungsvariablen `GAUSSIAN_OPT`, `SOBEL_OPT`, `NMS_OPT` und `HYSTERESIS_OPT` als `-D`-Flags weiter, zum Beispiel:

```
GAUSSIAN_OPT=1 SOBEL_OPT=0 NMS_OPT=1 HYSTERESIS_OP=3 ./build_release.sh
```

Ohne Angabe greifen die Standardwerte, die im Script angegeben werden, wobei meist die am weitesten optimierte Variante (inkl. Pinned Memory) gewählt wurde, während `WARMUP` (Standard: aktiviert) einen Aufwärm-Kerneldurchlauf vor der Zeitmessung steuert. Da separable Device-Compilation (`-rdc=true`) verwendet wird, ist zudem ein Device-Link-Schritt (`nvcc -dlink`) vor dem finalen Host-Link nötig, bei dem die Ziel-Architektur

(`-arch=sm_75` für RTX 2060, `-arch=sm_120` für die RTX 5070 Ti) erneut angegeben werden muss, da `nvcc` sonst implizit auf eine ältere Architektur zurückfällt.

Neben den genannten Parametern kann für die Kompilierung via `FUSED_PIPELINE=1` auch die Variante mit reduzierten $D \rightarrow H/H \rightarrow D$ Transfers zwischen den einzelnen Schritten aktiviert werden sowie via `BENCHMARK_RUNS` die Anzahl der Kernel-Runs übergeben werden.

Analog erzeugt `build_debug.sh` eine Debug-Variante mit den Flags `-G -O0`, ohne `-use_fast_math`; alle berichteten Messungen stammen ausschließlich aus dem Release-Build. Das Ausführen der jeweils gebauten Binary übernehmen die separaten Skripte `run_release.sh` bzw. `run_debug.sh` mit fest hinterlegtem Ein- und Ausgabepfad.

1.5 Messmethodologie

Die Performance-Messungen wurden unter nicht-isolierten Bedingungen durchgeführt: Hintergrundprozesse, thermisches Throttling, dynamisches Power Management der Laptop-GPU, nicht garantiert geleerte Caches sowie OS-Scheduling-Interrupts bei $H \rightarrow D/D \rightarrow H$ -Transfers können die Ergebnisse beeinflusst haben. Die Ergebnisse sind daher als Richtwerte zu verstehen. Um Schwankungen zu minimieren, wurde jede Messung als Durchschnitt über 200 Runs ermittelt, wobei der erste Aufruf (Warm-up Run) separat ausgewiesen wird, da CUDA-Kontext und Page-Cache zu diesem Zeitpunkt noch nicht aufgewärmt sind. NCU-Profilng wurde mit 10 Kernel-Instanzen pro Report durchgeführt, der Warm-up Run wurde dabei ausgeschlossen. Gemessen wurde lediglich mit einem Bild, das allerdings relativ groß ist und eine realistische Beispiel für Bildbearbeitung darstellt (5120x2880 Pixel). Wir sind uns bewusst, dass dies prinzipiell problematisch ist und eine erweiterte Analyse auch andere Bildgrößen beinhalten sollte.

Abseits des Gaussian Blur Schritts, der nur von Matthias Stefan gemessen wurde, haben wir alle drei weiteren Schritte auf beiden Systemen gemessen. Auch wurde von uns beiden die Laufzeit der gesamten Pipeline gemessen (sowohl die Fused Pipeline als auch die nicht-optimierte Version mit vielen $H \rightarrow D/D \rightarrow H$ Transfers). Die Messdaten sind im Appendix beigefügt.

Neben der Laufzeitmessung, die innerhalb des Programmcodes verortet ist, wurde außerdem NCU verwendet. Dazu wurden zuerst alle Messwerte erhoben und diese folgend mit einem Python Script auf eine Auswahl reduziert. Für die Messung wurde folgender Befehl verwendet (hier beispielhaft für die Messung des naiven Sobel Kernels):

```
sudo ncu -set full -k regex:naive_sobel_filter \  
-s 0 -c 11 -f -o analysis/sobel_naive.ncu-rep \  
./bin/canny_release.out ./assets/maple-leaf-black-5120x2880.jpg \  
/tmp/sobel-profile.png
```

Die Messung des Warmup-Runs wurde später im Python Script herausgefiltert.

1.6 Verwandte Arbeiten

Dieser Abschnitt stellt vier Arbeiten vor, die den Canny-Algorithmus auf GPU- bzw. GPU-nahen Architekturen parallelisieren, mit besonderem Fokus darauf, welche Speicheroptimierungen die jeweiligen Autoren für die Non-Maximum Suppression einsetzen und begründen.

Der in diesem Praktikum implementierte Algorithmus basiert auf dem von Canny [1] in *A Computational Approach to Edge Detection* vorgeschlagenen mehrstufigen Verfahren zur Kantendetektion, bestehend aus Gaussian Blur, Gradientenberechnung, Non-Maximum Suppression und Hysteresis Thresholding, dem die eigene Pipeline unmittelbar folgt. Eine gut lesbare Einführung in die Funktionsweise von Canny Edge Detection im Gesamtkontext von Edge Detection findet sich im Buch *Digital Image Processing* [2, S. 729ff.] im Kapitel zu „Point, Line, and Edge Detection“. Von dort wurden auch einige Optimierungen der Formeln entnommen, beispielsweise für den Sobel Filter.

Song et al. [5] implementieren in *A Parallel Canny Edge Detection Algorithm Based on OpenCL Acceleration* eine OpenCL-basierte Variante und begründen den Einsatz von Local Memory ausschließlich für die Faltungsstufen, da hier pro Pixel n^2 Nachbarn gelesen werden; für die Non-Maximum Suppression, die nur zwei Nachbarn pro Pixel benötigt, fehlt im Pseudocode dagegen jeder Lade- oder Kachelungsschritt.

Ogawa et al. [4] stellen in *Efficient Canny Edge Detection using a GPU* eine frühe CUDA-Implementierung vor, bei der die Non-Maximum Suppression ohne Kachelung direkt auf dem Global Memory arbeitet. Der eigentliche Schwerpunkt liegt auf einer stack-basierten Traversierung der nachgelagerten Hysteresis-Stufe, die im Gegensatz zu Ansätzen mit fester Traversierungstiefe auch lange, über mehrere Bildkacheln verlaufende Kantenketten vollständig erfasst.

Horvath et al. [3] implementieren in *Canny Edge Detection on GPU using CUDA* die Canny-Pipeline mit einem eigenen Kernel pro Stufe. Die Shared-Memory-Implementierung der Non-Maximum-Suppression orientiert sich am Tiling-Ansatz des Sobel-Kernels, benötigt aufgrund der geringeren Nachbarschaftsgröße jedoch nur einen minimalen Halo-Rand. Mittels NSIGHT Compute identifizieren die Autoren zudem `hypot/atan2` als Hauptbottleneck ihres Sobel-Kernels; deren Ersatz durch konstante Zuweisungen reduzierte die Kernel-Laufzeit um den Faktor 8.0.

Yadav und Gupta [6] vergleichen in *Optimizing CUDA Kernels for High-Performance Canny Edge Detection* vier Optimierungsstufen anhand einer Roofline-Analyse, die sich jedoch ausschließlich auf den Gaussian-Filter-Kernel bezieht. Bemerkenswert ist, dass die Autoren die Non-Maximum Suppression konzeptionell nicht dem Shared-Memory-Ansatz, sondern der Warp-Level-Optimierung zuordnen, ohne diesen Kernel jedoch gesondert zu profilieren.

2 Algorithmus

2.1 Überblick

Der Canny-Algorithmus ist ein mehrstufiges Verfahren zur Kantendetektion in Graustufenbildern. Canny definiert drei Kriterien für einen optimalen Kantendetektor: gute Detektion, präzise Lokalisierung sowie exakt eine Antwort pro Kante, die auf eine Breite von einem Pixel supprimiert werden soll [1, S. 680]. Die Verarbeitung erfolgt in vier aufeinanderfolgenden Schritten, wobei der Output jedes Schritts als Input des nächsten dient (siehe Abbildung 1). Die einzelnen Schritte werden in den kommenden Abschnitten näher erläutert.

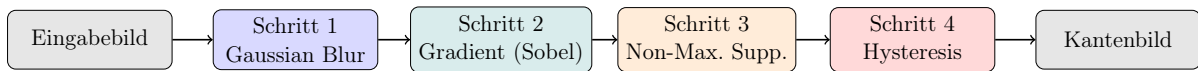


Abbildung 1: Canny Edge Detection Pipeline

2.2 Schritte

2.2.1 Schritt 1: Gaussian Blur

Um einen mathematisch optimalen Kantendetektor herzuleiten, trifft Canny eine Annahme über das Rauschen im Bild: „*We will assume that the image consists of the edge and additive white Gaussian noise*“ [1, S. 680]. Da dieses Rauschen fälschlicherweise als Kante detektiert werden kann, muss das Bild zunächst geglättet werden. Canny zeigt, dass der optimale Filter für dieses Rauschmodell durch die erste Ableitung einer Gaußfunktion approximiert werden kann [1, S. 687]:

$$G(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \quad (1)$$

Der Parameter σ kontrolliert die Stärke der Glättung. Zwischen Rauschunterdrückung und Lokalisierungsgenauigkeit besteht jedoch ein grundlegender Zielkonflikt, den Canny als „*uncertainty principle*“ zwischen Detektion und Lokalisierung bezeichnet [1, S. 684].

2.2.2 Schritt 2: Sobel

Nach dem Gauß-Filter folgt der Sobel Schritt, der es als Ziel hat, mit Hilfe des Sobel Kernels [2, S. 716ff.] für jeden Bildpunkt die Stärke und Richtung der stärksten Intensitätsänderung zu bestimmen, da diese durch den starken Kontrast auf Kanten hinweisen können.

Der Sobel Filter nutzt dabei zwei 3×3 -Faltungskerne, die die partiellen Ableitungen der Bildintensität I in x - und y -Richtung approximieren:

$$G_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix} * I, \quad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{pmatrix} * I \quad (2)$$

Details zu diesen Kernen und weitere Kernel wie den Prewitt-Kernel finden sich in der Literatur [2] und sollen an dieser Stelle nicht weiter besprochen werden.

Die Grundidee ist, dass aus den mit Hilfe der Kernel für jeden Bildpunkt (x, y) berechneten Werten $G_x(x, y)$ und $G_y(x, y)$ die Gradientenstärke

$$G(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2} \quad (3)$$

sowie die Gradientenrichtung

$$\theta(x, y) = \text{atan2}(G_y(x, y), G_x(x, y)) \quad (4)$$

berechnet werden. Für diese Formeln können jedoch auch effizientere Alternativen gewählt werden (siehe die folgende Sektion zur Implementierung und Optimierung). Die Verwendung von atan2 anstelle von $\arctan(G_y/G_x)$ ist hierbei essenziell, da atan2 beide Vorzeichen von G_x und G_y getrennt berücksichtigt und dadurch den korrekten Winkel im vollen Bereich $(-\pi, \pi]$ liefert, während der einfache Arkustangens aufgrund seines eingeschränkten Wertebereichs von $(-\pi/2, \pi/2)$ zwischen gegenüberliegenden Quadranten nicht unterscheiden kann.

Wichtig ist, dass der Gradientenvektor (G_x, G_y) in die Richtung des stärksten Intensitätsanstiegs zeigt, was aber nicht der Kantenrichtung entspricht (diese steht senkrecht zur Richtung der Intensitätsänderung). Die Kantenorientierung selbst ergibt sich somit als $\theta(x, y) + \frac{\pi}{2}$ (bzw. äquivalent $\theta(x, y) - \frac{\pi}{2}$).

Für die Non-Maximum-Suppression wird üblicherweise direkt mit der Gradientenrichtung θ gearbeitet: Sie wird auf einen von vier diskreten Sektoren ($0^\circ, 45^\circ, 90^\circ, 135^\circ$) quantisiert, um zu bestimmen, mit welchen Nachbarpixeln entlang der Gradientenrichtung die Magnitude verglichen werden muss, um lokale Maxima zu identifizieren und Kanten auf eine Breite von einem Pixel zu reduzieren.

Am Ende des Sobel Schritts liegen somit zwei Matrizen vor, mit denen die weiteren Schritte durchgeführt werden: Eine Matrix mit den Gradientenstärken jedes Bildpunkts (wie stark ändert sich die Intensität im Vergleich zu den Nachbarpixeln?) und eine Matrix mit den Richtungen der Kanten dieser Intensitätsänderungen (was *nicht* der Richtung des Gradienten entspricht!).

2.2.3 Schritt 3: Non-Maximum Suppression

Schritt 2 liefert für jeden Pixel einen Gradientenbetrag, der die lokale Kantenstärke beschreibt, wobei die resultierenden Kanten an vielen Stellen breiter als ein Pixel sind. Non-Maximum Suppression soll diese breite Gradientenantwort im Idealfall auf eine Breite von einem Pixel reduzieren, indem entlang der Gradientenrichtung nur das lokale Maximum erhalten bleibt. Canny begründet diese Forderung mit dem Kriterium präziser Lokalisierung: „*The points marked as edge points by the operator should be as close as possible to the center of the true edge*“ [1, S. 680]. Formal entspricht der gesuchte Kantenpunkt einer Nullstelle der zweiten Richtungsableitung des geglätteten Bildes entlang der Gradientenrichtung n [1, S. 691], die aus der geglätteten Gradientenrichtung geschätzt wird, da sie nicht vorab bekannt ist.

In der diskreten Implementierung wird dies approximiert, indem die Gradientenmagnitude jedes Pixels mit den beiden direkten Nachbarn entlang der auf vier Klassen ($0^\circ/45^\circ/90^\circ/135^\circ$)

quantisierten Gradientenrichtung verglichen wird. Ein Pixel überlebt die Suppression genau dann, wenn seine Magnitude größer oder gleich beiden Nachbarn ist, andernfalls wird er auf null gesetzt. Die Ein-Pixel-Breite wird dabei nur bei einem strikten lokalen Maximum tatsächlich erreicht, denn bei einem Plateau, also mehreren benachbarten Pixeln mit identischer Magnitude, erfüllen aufgrund des $\geq$ -Vergleichs alle beteiligten Pixel gleichzeitig das Überlebenskriterium, wodurch die Kante an dieser Stelle breiter als ein Pixel bleibt.

2.2.4 Schritt 4: Hysteresis Thresholding

Nach der Non-Maximum Suppression liegt ein Bild vor, in dem Kanten bereits auf eine Breite von einem Pixel reduziert wurden, das jedoch noch immer Kantenpunkte unterschiedlichster Stärke enthält – darunter auch solche, die auf Bildrauschen oder schwachen, irrelevanten Intensitätsschwankungen beruhen. Der letzte Schritt des Canny-Algorithmus, das Hysteresis-Thresholding, dient dazu, aus dieser Menge an Kandidaten die tatsächlich relevanten Kanten zu extrahieren.

Der Canny-Algorithmus löst dieses Problem durch die Einführung zweier Schwellwerte, eines oberen Schwellwerts T_{high} und eines unteren Schwellwerts T_{low} , wobei üblicherweise $T_{\text{high}} = 2 \cdot T_{\text{low}}$ bis $T_{\text{high}} = 3 \cdot T_{\text{low}}$ gewählt wird [2, S. 732.]. Anhand dieser beiden Schwellwerte werden alle Kantenpixel in drei Kategorien eingeteilt:

- **Starke Kanten** ($G(x, y) \geq T_{\text{high}}$): Diese Pixel werden unmittelbar als sichere Kantenpunkte übernommen.
- **Schwache Kanten** ($T_{\text{low}} \leq G(x, y) \leq T_{\text{high}}$): Diese Pixel sind Kandidaten, deren endgültige Klassifikation von ihrer Nachbarschaft zu starken Kanten abhängt.
- **Kein Kantenpixel** ($G(x, y) < T_{\text{low}}$): Diese Pixel werden sofort und endgültig verworfen.

Der eigentliche Hysteresis-Algorithmus besteht darin, dass ein schwacher Kantenpixel nur dann übernommen wird, wenn er über eine Kette benachbarter Pixel (üblicherweise 8-Konnektivität, also die direkten Nachbarn) mit mindestens einem starken Kantenpixel verbunden ist. Dies kann entweder in einem einzigen Schritt geschehen oder aber iterativ, um Ketten von starken Kantenpixeln basierend auf vorherigen Schritten zu kreieren. Da in [2] nur ein Durchlauf besprochen wird, wird auch die Implementierung in dieser Arbeit nur einen Schritt durchführen. Es wäre aber möglich, auch iterativ vorzugehen und den Hysteresis-Schritt mehrfach durchlaufen zu lassen, bis sich keine Änderungen mehr ergeben (sprich keine schwachen Pixel mehr in starke umgewandelt werden). Dies kann entweder durch eine lineare Suche geschehen, indem ausgehend von einem starken Pixel die Nachbarschaft abgesucht wird, und dann jeder weitere starke Pixel bzw. schwache Pixel (der ja nun als stark markiert wurde durch seine Nachbarschaft zu einem starken Pixel) in der Nachbarschaft wiederum als Ausgangspunkt für eine neue Suche genommen wird. Dieses Vorgehen ist jedoch nicht gut parallelisierbar, weshalb für diese Arbeit eine andere, deutlich reduzierte Implementierung gewählt wurde (siehe den entsprechenden Abschnitt im Implementierungskapitel).

3 Implementierung

3.1 Schritt 1: Gaussian Blur

3.1.1 Naive Implementierung

Die Gaußgewichte werden auf der CPU vorberechnet und normalisiert (Summe 1, Gleichung 1), einmalig per `cudaMemcpy` auf die GPU übertragen und stehen dort als Read-only-Buffer im globalen Speicher zur Verfügung. Abbildung 2 zeigt das resultierende 3×3 Gewichtsrastrer für $\sigma = 1.0$.

0.0894	0.1201	0.0894
0.1201	0.1615	0.1201
0.0894	0.1201	0.0894

 $\times$

L10	L20	L30
L50	L48	L10
L90	L30	L20

 $=$

L34

Abbildung 2: Gaussian Blur, gewichteter Mittelwert am Beispieldpixel $L48 \rightarrow L34$

Im CUDA-Kernel `gaussian_filter()` ist jeder Thread für genau einen Ausgabepixel zuständig; Threads außerhalb der Bildgrenzen werden sofort verworfen. Jeder Thread iteriert über alle $(2r+1)^2$ Nachbarpixel, liest jeden Wert direkt aus dem globalen Speicher und akkumuliert die gewichtete Summe, wobei Pixel außerhalb der Bildgrenzen mit 0 aufgefüllt werden (Zero-Padding). Die Threads eines Blocks greifen dabei in y-Richtung auf Adressen mit Abstand `width` zu, wodurch die Zugriffe auf den globalen Speicher nicht coalesced erfolgen, was die in Abschnitt 3.1.2 beschriebene Shared Memory Optimierung motiviert; die quantitative Auswertung findet sich in Abschnitt 4.1.

3.1.2 Shared Memory Optimierung

Bei einem Kernel der Größe $(2r+1)^2$ liest jeder Thread bis zu $(2r+1)^2$ Pixel redundant aus dem DRAM. [3, S. 2] adressieren dieses Problem durch Shared Memory Tiling, bei dem Threads eines Blocks kooperativ eine Bildkachel inklusive Halo-Rand der Größe $(\text{BLOCK_SIZE} + 2 \cdot \text{BLUR_RADIUS})^2$ in den On-Chip Speicher laden, bevor `__syncthreads()` die vollständige Ladephase sicherstellt. Die Gaußgewichte werden zusätzlich in `__constant__` Memory ausgelagert, da alle Threads dieselben Gewichte lesen und dieser Speicher für uniforme Zugriffe hardwareseitig gecacht wird. Die Auswirkung dieser Optimierung auf Laufzeit und Speicherdurchsatz wird in Abschnitt 4.1 ausgewertet.

3.1.3 Pinned Memory Optimierung

Pinned Memory (Page-locked Memory) ist Host-seitiger Speicher, der vom Betriebssystem nicht ausgelagert werden kann, wodurch direkte DMA-Transfers ohne zwischengeschalteten CPU-Cache möglich sind. Die Änderung gegenüber der Shared Memory Implementierung beschränkt sich auf die Host-seitige Allokation (`cudaMallocHost()` statt `malloc()`); der Kernel bleibt identisch, der erwartete Vorteil zeigt sich primär beim Device-to-Host-Transfer (Abschnitt 4.1).

3.1.4 Warp-Level Optimierung

Als vierte Stufe wurden Warp-Level Intrinsics (`__shfl_sync()`) evaluiert, die Datenaustausch innerhalb eines Warps ohne Shared Memory ermöglichen. Für 2D-Stencil-Operationen wie den Gaussian Blur ist dieser Ansatz jedoch ungeeignet, da `__shfl_sync()` primär für 1D-Reduktionen konzipiert ist, was [6] bestätigen (79.29 % vs. 83.33 % Compute/Memory Throughput bei Shared Memory). Eine eigene Implementierung wurde daher nicht vorgenommen.

3.2 Schritt 2: Sobel

Die Sobel-Stufe erhält das geglättete Graustufenbild aus dem vorhergehenden Gaussian-Schritt und berechnet für jeden Pixel den horizontalen und vertikalen Gradienten. Dazu wird ein zweidimensionales Grid verwendet, wobei jeder Thread immer für einen Bildpixel zuständig ist. Die Ausgaben der Sobel-Stufe sind der Gradientenwert sowie die Gradientenrichtung, die anschließend von der Non-Maximum Suppression verwendet werden.

3.2.1 Naive Implementierung

Die naive Variante `naive_sobel_filter()` lädt für jedes innere Pixel seine 3×3 -Nachbarschaft aus dem globalen Speicher. Die beiden Sobel-Filtermasken lauten

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}. \quad (5)$$

Für jeden Nachbarschaftspixel werden die Berechnungen mit beiden Masken aufsummiert. Der Gradientenbetrag wird anschließend als

$$M = \sqrt{G_x^2 + G_y^2} \quad (6)$$

berechnet, auf den 8-Bit-Bereich $[0, 255]$ begrenzt und als `uint8_t` gespeichert. Die Richtung wird mit `atan2f( $G_y$ ,  $G_x$ )` in Radiant bestimmt, wobei es sich hierbei um eine importierte Bibliotheksfunktion handelt. Da ein Sobel-Filter für jeden Ausgabepixel dieselben Nachbarpixel wie umliegende Threads liest, entstehen bei dieser Variante viele redundante Speicherzugriffe.

```
1 int gx = 0, gy = 0;
2 for (int i = -1; i <= 1; ++i) {
3     for (int j = -1; j <= 1; ++j) {
4         const uint8_t pixel = src_buffer[(y + i) * width + (x + j)];
5         gx += pixel * sobelX[i + 1][j + 1];
6         gy += pixel * sobelY[i + 1][j + 1];
7     }
8 }
9
10 float gradient = sqrtf(gx * gx + gy * gy);
11 gradient = fminf(255.0f, fmaxf(0.0f, gradient));
12 out_grad_buffer[y * width + x] = static_cast<uint8_t>(gradient);
13 out_dir_buffer[y * width + x] = atan2f(float(gy), float(gx));
```

Listing 1: Kernschritte der naiven Sobel-Implementierung

3.2.2 Shared-Memory-Variante

Die Variante `sharedm_sobel_filter()` reduziert die redundanten Speicherzugriffe durch Tiling. Die Threads eines Blocks laden gemeinsam eine Bildkachel in den Shared Memory. Da der Sobel-Operator einen Radius von eins besitzt, wird die Kachel an jeder Seite um einen Pixel als Halo erweitert. Threads an den Blockgrenzen laden dabei die zusätzlichen Randpixel. Für Koordinaten außerhalb des Bildes wird Zero-Padding verwendet. Die mathematische Berechnung von Betrag und Richtung entspricht der naiven Variante.

Für diese Variante wird testweise ein anderes Blocklayout verwendet: Die Kacheln werden mit einem Block von `BLOCK_SIZE_X × BLOCK_SIZE_Y` Threads verarbeitet, während die übrigen Varianten quadratische Blöcke verwenden. Dadurch lässt sich das Ladeverhalten der Kachel unabhängig von der Berechnung untersuchen. Die Idee war hier, dass durch eine andere Block-Size wie 32 x 8 versucht werden kann, beim Laden aus dem globalen Speicher eine besseres Coalescing zu erreichen. Da dies allerdings zu keinen Unterschieden in der Laufzeit geführt hat, wurde dieser Ansatz wieder verworfen und letztlich fast immer auf die 16x16 Blockgröße zurückgegriffen.

3.2.3 Arithmetische Optimierung

Da die Shared-Memory-Variante in den Messungen leider nie schneller war, verwendet `optimized_sobel_filter()` einen anderen Ansatz, indem hier vor allem die vorherigen Loop-Zugriffe inklusive der Zugriffe auf Arrays mit den entsprechenden Werten für die Faltungen vollständig entrollt wurden. Multiplikationen wurden zudem als Bitverschiebungen formuliert. Zusätzlich wird der Betrag durch die L1-Norm

$$M_{L1} = |G_x| + |G_y| \quad (7)$$

angenähert, was ebenfalls als Optimierung in [2] vorgeschlagen wurde. Diese Berechnung vermeidet die teure Quadratwurzel, ist aber nicht identisch zur naiven Variante. Die Richtung wird in dieser Variante weiterhin mit der teuren `atan2f()` Bibliotheksfunktion bestimmt.

3.2.4 Richtungs-Buckets

Die Variante `bucket_sobel_filter()` baut auf der optimierten Variante (Arithmetische Optimierung) auf, vermeidet aber zusätzlich `atan2f()`, was möglich ist, weil für die Non-Maximum Suppression nur vier ungerichtete Richtungen anstelle der genauen Radianten-Werte benötigt werden: 0°, 45°, 90° und 135°. Anhand des Verhältnisses $|G_y|/|G_x|$ und des Vorzeichens der beiden Komponenten wird die Richtung direkt in einen Bucket mit dem Wert 0 bis 3 eingeordnet. Dadurch wird die Richtung statt als `float` als `uint8_t` gespeichert, und die nachfolgende NMS muss die Quantisierung nicht erneut berechnen.

3.2.5 Split-Variante

Als letzte Sobel-Konfiguration wurden die beiden Filterkomponenten für die Gradientenberechnung separat berechnet. `split_sobel_x_filter()` und `split_sobel_y_filter()` schreiben G_x und G_y in temporäre `int16_t`-Puffer. Ein dritter Kernel, `split_sobel_combine_filter()` berechnet daraus die L1-Magnitude und den Richtungs-Bucket. Diese Variante benötigt somit drei Kernelstarts und zusätzlichen Gerätespeicher.

3.3 Schritt 3: Non-Maximum Suppression

3.3.1 Naive Implementierung

Die Non-Maximum Suppression wird im CUDA-Kernel `non_maximum_suppression()` umgesetzt, wobei jeder Thread für genau einen Pixel zuständig ist. Als Input erhält der Kernel den Gradientenbetrag (`magnitude_buffer`) sowie die Gradientenrichtung in Radiant (`direction_buffer`) aus Schritt 3.2.

Die Gradientenrichtung wird zunächst von Radiant in Grad umgerechnet und auf den Bereich $[0^\circ, 180^\circ)$ gefaltet, da eine Kante keine Orientierung besitzt und z. B. 170° und -10° dieselbe Achse beschreiben. Anschließend wird die Richtung auf eine von vier Klassen quantisiert, die den vier möglichen Achsen im diskreten Pixelraster entsprechen (Abbildung 3). Je nach Richtungsklasse werden die Offsets (dx, dy) zum Nachbapixel bestimmt; der Pixel überlebt die Suppression genau dann, wenn seine Magnitude größer oder gleich beiden Nachbarn entlang dieser Achse ist, andernfalls wird er auf null gesetzt (Listing 2).

```
1  __global__ void non_maximum_suppression(  
2      const uint8_t* magnitude_buffer,  
3      const float* direction_buffer,  
4      const int32_t width,  
5      const int32_t height,  
6      uint8_t* out_nms_buffer)  
7  {  
8      // Thread-Position bestimmen, out-of-bounds Threads verwerfen  
9      ...  
10  
11     const uint8_t magnitude = magnitude_buffer[y * width + x];  
12     const float direction = direction_buffer[y * width + x];  
13  
14     // Richtung von Radiant in Grad umrechnen und auf [0, 180) falten  
15     ...  
16  
17     // Richtung auf eine von vier Klassen quantisieren, Offset (dx, dy) bestimmen  
18     int32_t dx, dy;  
19     if (deg < 22.5f) { dx = 1; dy = 0; }  
20     else if (deg < 67.5f) { dx = 1; dy = -1; }  
21     else if (deg < 112.5f) { dx = 0; dy = 1; }  
22     else if (deg < 157.5f) { dx = 1; dy = 1; }  
23     else { dx = 1; dy = 0; }  
24  
25     const uint8_t neighbor_a =  
26         sample_clamped(magnitude_buffer, width, height, x + dx, y + dy);  
27     const uint8_t neighbor_b =  
28         sample_clamped(magnitude_buffer, width, height, x - dx, y - dy);  
29  
30     out_nms_buffer[y * width + x] =  
31         (magnitude >= neighbor_a && magnitude >= neighbor_b) ? magnitude : 0;  
32 }
```

Listing 2: `non_maximum_suppression` Kernel (gekürzt)

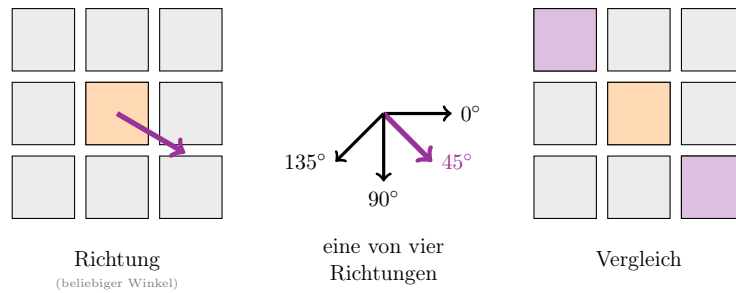


Abbildung 3: Non-Maximum Suppression: Die Gradientenrichtung wird auf eine von vier Achsen quantisiert (0° , 45° , 90° , 135°). Anschließend wird der Pixel mit den beiden farbig markierten Nachbarn entlang der gewählten Achse verglichen.

3.3.2 Shared Memory Optimierung

Die Shared-Memory-Variante lädt zunächst kooperativ eine Bildkachel inklusive eines Halo-Rands der Breite 1 in den On-Chip Speicher, statt für jeden Nachbarn einen eigenen Zugriff auf den globalen Speicher auszuführen. Der Halo-Rand fällt dabei deutlich schmäler aus als bei Gaussian Blur (Abschnitt 3.1.2), da Non-Maximum Suppression stets nur die unmittelbare 8er-Nachbarschaft benötigt. Da alle Threads eines Blocks am kollektiven Laden beteiligt sein müssen, erfolgt die Grenzprüfung erst nach `__syncthreads()`, um einen Deadlock durch nicht erreichte Threads zu vermeiden. Magnitude und Nachbarn werden anschließend direkt aus der Kachel statt aus dem globalen Speicher gelesen. Die quantitative Auswertung findet sich in Abschnitt 4.3.

3.3.3 Bucket-Optimierung

Als letzte Optimierungsmöglichkeit verwendet diese Stufe einen vom Sobel-Kernel bereits vorquantisierten Richtungs-Bucket $\in \{0, 1, 2, 3\}$ statt Radiant-Werten, wodurch Gradumrechnung und Klassenbestimmung entfallen; die (dx, dy) -Offsets kommen stattdessen aus einer Lookup-Tabelle im Constant Memory. Die Magnitude bleibt gekachelt, da sie mehrfach gelesen wird; der Bucket-Wert dagegen wird direkt aus dem globalen Speicher gelesen, da hier keine Datenwiederverwendung zwischen Threads stattfindet.

3.4 Schritt 4: Hysteresis

Die Hysteresis-Stufe erhält das von der Non-Maximum Suppression ausgedünnte Bild. In der Implementierung werden die beiden Schwellwerte `low_t` und `high_t` mit `low_t = 50` und `high_t = 150` initialisiert (was dem empfohlenen 3:1 Verhältnis entspricht).

Wie bereits in der Sektion zur Erklärung des Sobel-Schritts ausgeführt, wird in dieser Implementierung kein iteratives Vorgehen, sondern nur eine einfache Einzeldurchlaufsvariante implementiert mit nur einem Kernel-Start. Eine iterative Variante würde zusätzliche Abhängigkeiten zwischen den Durchläufen und globale Synchronisationen benötigen.

```

1  const uint8_t p = src_buffer[index];
2  uint8_t out = 0;
3
4  if (p >= high_threshold) {
5      out = 255;                                     // starke Kante
6  } else if (p >= low_threshold) {
7      uint8_t max_n = 0;
8      // Maximum der gueltigen 8 Nachbarn bestimmen

```

```

9      for (int dy = -1; dy <= 1; ++dy)
10         for (int dx = -1; dx <= 1; ++dx)
11             if ((dx != 0 || dy != 0) && valid(x + dx, y + dy))
12                 max_n = max(max_n, src_buffer[(y + dy) * width + x + dx]);
13     out = (max_n >= high_threshold) ? 255 : 0;
14 }
15
16 out_buffer[index] = out;

```

Listing 3: Ein-Schritt-Klassifikation im Hysteresis-Kernel

3.4.1 Naive Implementierung

Im Kernel `naive_hysteresis_linker()` berechnet jeder Thread die globale Position eines Pixels. Starke Pixel werden ohne Nachbarschaftszugriff direkt akzeptiert. Für schwache Pixel werden bis zu acht Nachbarn aus dem globalen Speicher gelesen und auf den oberen Schwellwert geprüft. Sobald ein starker Nachbar gefunden wurde, kann die Suche beendet werden. Das Ergebnis wird in einen separaten 8-Bit-Ausgabepuffer geschrieben.

3.4.2 Shared-Memory-Optimierung

`hysteresis_sharedm()` lädt zunächst gemeinsam mit allen Threads eines Blocks eine Kachel in den Shared Memory. Weil jeder Pixel seine unmittelbare Nachbarschaft benötigt, wird die Kachel an jeder Seite um einen Pixel als Halo erweitert. Randthreads laden die Seiten und Ecken des Halos; außerhalb des Bildes liegende Positionen werden mit 0 gefüllt. Die acht Nachbarn eines schwachen Pixels werden dann aus dem Shared Memory gelesen, wodurch dieselben Global-Memory-Werte nicht für jeden Nachbarschaftszugriff erneut geladen werden müssen.

3.4.3 Optimierte Variante und Randbehandlung

Die Variante `hysteresis_opt()` verzichtet auf Shared-Memory-Tiling (weil dieses auch hier wie im Sobel-Schritt keine Laufzeitverbesserung gebracht hat). Für innere Pixel werden die acht Nachbarzugriffe stattdessen vollständig entrollt. Das reduziert Schleifen- und Indexierungs-Overhead.

Bei Randpixeln sind nicht alle acht Nachbarschaftspositionen gültig. Deshalb verwendet der optimierte Kernel für innere Pixel den schnellen unrolled Pfad und für Randpixel eine separate, bounds-geprüfte Nachbarschaftsschleife. So werden weder negative Indizes noch Zugriffe hinter dem Bildende erzeugt, während schwache Randpixel weiterhin von tatsächlich vorhandenen starken Nachbarn übernommen werden können.

4 Ergebnisse und Analyse

4.1 Gaussian Blur

4.1.1 Korrektheit

Abbildung 4 zeigt das Ergebnis der Filterung im Vergleich zum Originalbild.



Abbildung 4: Vergleich Original (links) vs. Gaussian Blur (rechts, $\sigma = 25$, `BLUR_RADIUS=12`). Die Parameter wurden bewusst extrem gewählt, um den Effekt deutlicher sichtbar zu machen, in der Pipeline wird ein deutlich sanfterer Filter verwendet.

4.1.2 Performance

Das NCU-Profilung der naiven Implementierung zeigt, dass 47 % aller Global Memory Zugriffe uncoalesced sind, was die in Abschnitt 3.1.2 beschriebene Shared Memory Optimierung motiviert.

Der gemessene Speedup hängt stark vom `BLUR_RADIUS` ab (Abbildung 5). Bei $r = 2$ sinkt die Kernel-Laufzeit trotz nahezu identischem SM Throughput bereits um 40.7 % ($1.69\times$), bei $r = 9$ verstärkt sich dieser Effekt auf einen Speedup von $3.0\times$ bei deutlich gesteigertem SM Throughput, da hier die 361 Faltungsoperationen pro Pixel den redundanten DRAM-Traffic zum dominanten Faktor machen.

Die Pinned-Memory-Variante bestätigt, dass Compute Throughput und Occupancy bei $r = 9$ praktisch identisch zur Shared Memory Variante sind, da der Kernel selbst unverändert bleibt. Der Vorteil zeigt sich ausschließlich im D→H-Transfer, der bei $r = 9$ von 2.04 ms auf 1.37 ms sinkt. Die vollständigen Messdaten sind in Anhang A (S. 26) aufgeführt.

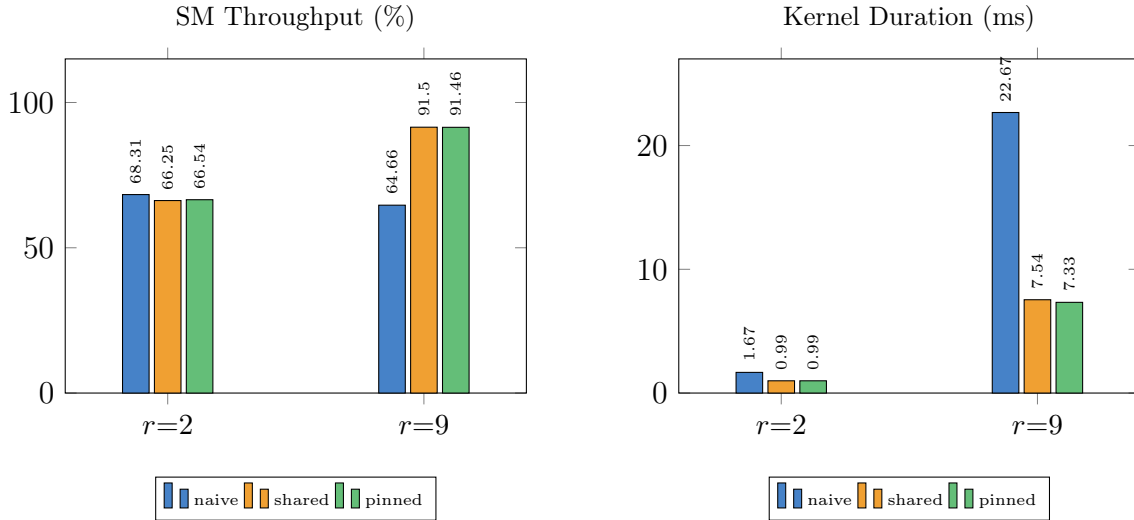


Abbildung 5: NCU-Profilng, Vergleich naive/shared/pinned bei $r=2$ und $r=9$ (200 Runs Avg).

4.2 Schritt 2: Sobel

4.2.1 Performance

Die folgende Auswertung verwendet die TJ-Messwerte aus dem Mittelwert von 200 Läufen. Der Warmup-Lauf wird nicht in den Vergleich einbezogen. Die vollständigen Messwerte sind in Anhang B aufgeführt.

Betrachtet werden die Varianten `naive`, `shared`, `split`, `pinned` und `bucket`. Die Variante `optimized` wird nicht zusätzlich aufgeführt, da `pinned` im Quellcode denselben `optimized_sobel_filter()`-Kernel verwendet. `pinned` ergänzt diesen Kernel lediglich einen locked Host-Buffer. Die `split` Variante enthält zusammengefasste Laufzeitwerte für alle drei Kernel.

Variante	H→D [ms]	Kernel [ms]	D→H [ms]	Gesamt [ms]
naive	0.528	0.140	9.649	10.317
shared	0.527	0.211	9.747	10.484
split	0.533	0.281	1.409	2.223
pinned	0.533	0.145	2.574	3.253
bucket	0.528	0.079	4.271	4.878

Tabelle 3: Sobel-Laufzeiten der betrachteten Varianten, TJ-Messung; Mittelwert aus 200 Läufen.

Wie anhand der Tabelle leicht erkennbar, hat die Reduzierung der Laufzeit des D→H Transfers durch Pinned Memory die größte Auswirkung auf die Laufzeit. Die separate Pinned Memory Funktion ist bei einigen Messungen bereits aktiviert (Split, Pinned, Bucket), was somit deren deutlich kürzere Gesamtlaufzeiten erklärt. Interessanter sind die Kernel-Laufzeiten. Auffällig ist, dass die Kernelzeit zwischen der naiven zur Shared Memory Variant von 0.140 ms auf 0.211 ms steigt. Das kooperative Laden der Kachel inklusive Halo sowie die notwendige Synchronisation erzeugen also bei dem kleinen 3×3 -Stencil mehr

Overhead als durch die Reduktion redundanter Memory-Zugriffe eingespart wird. Shared Memory bringt für diese Sobel-Ausführung daher keinen Laufzeitvorteil (verändert aber das Memory-Zugriffsverhalten, wie weiter unten in der NCU Analyse erkennbar wird).

Die **pinned**-Variante verwendet wie die arithmetisch optimierte Konfiguration den einzelnen `optimized_sobel_filter()`-Kernel. Dieser Kernel adressiert die drei Bildzeilen über Zeiger, entrollt die Sobel-Berechnung und verwendet Bitverschiebungen für die Multiplikation mit zwei. Die Kernelzeit von 0.145 ms liegt deshalb nahe an der naiven Variante (und unterschreitet diese auch häufig, durch die minimale Optimierung kann es hier jedoch zu Schwankungen in den Messergebnissen kommen), wird aber nicht durch Pinned Memory selbst verursacht. Außerdem verwendet der Kernel die L1-Norm $|G_x| + |G_y|$ statt der euklidischen Magnitude und ist daher nicht vollständig ergebnisidentisch zur naiven Variante.

Die **bucket**-Variante baut ebenfalls auf der entrollten Berechnung auf, ersetzt aber zusätzlich die teure Berechnung mit `atan2f()` durch eine direkte Einordnung von G_x und G_y in vier Richtungs-Buckets. Die Richtung wird dadurch als `uint8_t` statt als `float` gespeichert. Mit 0.079 ms besitzt **bucket** die kürzeste Kernelzeit, was deutlich macht, welchen großen Einfluss die Verwendung der `atan2f()` Bibliotheksfunktion auf die Laufzeit hat.

Die **split**-Variante berechnet G_x und G_y in zwei separaten Kernels und führt anschließend einen dritten Kombinationskernel aus. Dafür werden zwei temporäre `int16_t`-Puffer im Gerätespeicher angelegt. Die gemessene Kernelzeit umfasst alle drei Kernelstarts und beträgt daher 0.281 ms. Sie ist höher als bei allen anderen betrachteten Varianten. Trotzdem erreicht **split** mit 2.223 ms die kürzeste Gesamtlaufzeit. Ausschlaggebend sind der kleine Richtungsbuffer mit vier Buckets und der kurze D→H-Transfer von 1.409 ms.

Der Vergleich zeigt damit unterschiedliche Optimierungsziele. **split** ist in der vorliegenden Messung für die Gesamtzeit am günstigsten, während **bucket** den schnellsten einzelnen Kernel besitzt. **pinned** kombiniert den optimierten Kernel mit page-locked Host-Speicher und erreicht eine deutlich niedrigere Gesamtzeit als die naive und shared Versionen. Im Gegensatz zu **split** und **bucket** speichert **sobel_opt** (und damit **pinned**) die Richtung weiterhin als vollständigen Winkelwert aus `atan2f()`. Für die anschließende Non-Maximum Suppression reichen die vier Richtungs-Buckets aus; wenn jedoch die vollständige Winkelinformation benötigt wird, ist **pinned** die passendere Variante.

4.2.2 Kernelprofiling

Die zusammengefassten NCU Werte aus Anhang B ergänzen die Laufzeitmessung um die Auslastungs- und Cache-Metriken. Für **pinned** existiert kein eigener Profiling-Eintrag, da diese Variante denselben `optimized_sobel_filter()`-Kernel wie die optimierte Konfiguration verwendet. Der Eintrag **optimized** in der Profiling-Tabelle beschreibt daher den Kernel, der im Laufzeitvergleich durch **pinned** repräsentiert wird.

Metrik	naive	shared	optimized	bucket	split
Compute (SM) Throughput [%]	61.70	62.36	62.24	57.07	45.46
Memory Throughput [%]	53.36	37.52	54.02	57.13	56.23
DRAM Throughput [%]	53.36	34.06	54.02	28.73	44.00
Achieved Occupancy [%]	81.12	90.04	81.03	78.43	76.65
Warp Cycles per Issued Instruction	15.39	17.10	15.22	20.13	26.12
L1/TEX Hit Rate [%]	62.69	12.79	62.68	67.47	43.77
L2 Hit Rate [%]	68.43	39.65	68.73	82.03	65.26

Tabelle 4: Zusammengefasstes Sobel-Kernelprofiling aus Anhang B; zehn Runs je Variante, Warm-up ausgeschlossen.

Die Shared Memory Variante erreicht mit 90.04 % zwar die höchste Occupancy, weist aber gleichzeitig die niedrigste L1/TEX-Hit-Rate von 12.79 % und höhere Warp-Zyklen auf. Die höhere theoretische Auslastung führt deshalb nicht zu einer kürzeren Laufzeit, vielmehr scheinen die relativ hohen L1-Hitrates den Vorteil von Shared Memory zu nivellieren, weil bei dem kleinen Sobel-Faltungskernel die umliegenden Werte anscheinend meistens im L1-Cache vorliegen. Das Profiling liefert damit eine Erklärung, warum die Laufzeit der Shared Memory Variante die der naiven Variante nicht verbessert.

Insgesamt fällt auf, dass keiner der Kernel in Sachen Memory Throughput und Compute Throughput ausgelastet ist, die RTX 5070 Ti ist also mit dem relativ kleinen Bild unterfordert. Da der Compute Throughput meistens über dem des Memory Throughput liegt, kann davon ausgegangen werden, dass der Canny-Algorithmus hier Compute-Bound ist.

Der optimierte Kernel, der im Laufzeitvergleich über `pinned` betrachtet wird, zeigt gegenüber der naiven Variante nahezu identische Profilwerte. Compute Throughput, Memory Throughput, Occupancy, Cache-Hit-Raten und Warp-Zyklen ändern sich nur geringfügig. Die kürzere Laufzeit von `pinned` ist daher vor allem durch die Transferbedingungen und nicht durch eine deutlich höhere Kernel-Auslastung zu erklären.

Die Bucket-Variante besitzt mit 82.03 % die höchste L2-Hit-Rate und mit 57.13 % auch den höchsten Memory Throughput. Compute Throughput, DRAM Throughput und Occupancy liegen mit 57.07 %, 28.73 % und 78.43 % jedoch unter den Werten der naiven und optimierten Variante; außerdem fallen mehr Warp-Zyklen an. Ihre kurze Kernelzeit entsteht somit primär durch weniger Rechenarbeit und den kleineren Richtungsbuffer, nicht durch eine allgemein höhere GPU-Auslastung. Bei der Split-Variante fallen insbesondere die niedrige Occupancy von 76.65 % und die hohen Warp-Zyklen von 26.12 auf. Diese Werte sind mit dem zusätzlichen Aufwand durch die getrennten Gradienten-Kernels erklärbar.

4.3 Schritt 3: Non-Maximum Suppression

4.3.1 Korrektheit

Abbildung 6 zeigt einen Detailausschnitt des Ergebnisses der Non-Maximum Suppression im Vergleich zur Gradientenmagnitude aus Schritt 3.2: Die zuvor mehrere Pixel breite Gradientenantwort wird auf eine Breite von einem Pixel reduziert, mit Ausnahme lokaler Plateaus (vgl. Abschnitt 3.3).



Abbildung 6: Detailausschnitt: Vergleich Gradientenmagnitude (links) vs. Non-Maximum Suppression (rechts).

4.3.2 Performance

Das NCU-Profilung zeigt, dass der NMS-Kernel in allen drei Varianten sowohl beim Compute- als auch beim Memory-Throughput im Bereich von rund 31 % respektive 25 % verbleibt (Tabelle 15), der Kernel ist also weder rechen- noch bandbreitengebunden, sondern latenzgebunden. Jeder Thread liest lediglich seinen eigenen Magnitudenwert sowie die zwei Nachbarn entlang der Gradientenrichtung, weshalb der redundante DRAM-Traffic, der beim Gaussian Blur die Shared Memory Optimierung motiviert, hier weitgehend entfällt.

Entsprechend zeigt Abbildung 7 ein zum Gaussian Blur gegenläufiges Bild. Der SM Throughput bleibt über naive, shared und pinned praktisch unverändert, die Shared-Memory-Variante hebt die Auslastung also nicht an. Gleichzeitig steigt die Kernel-Laufzeit von 0.576 ms auf 0.879 ms, da das kooperative Laden der Kachel und die `__syncthreads()`-Barriere Overhead erzeugen, dem bei einem so kleinen Stencil keine nennenswerte Einsparung gegenübersteht.

Das Profiling bestätigt diese Deutung im Detail: Die Shared-Memory-Variante senkt zwar den DRAM Throughput von 26.15 % auf 17.43 % und hebt die Occupancy von 85.10 % auf 93.37 %, doch da der Kernel nicht bandbreitengebunden ist, übersetzt sich diese Einsparung nicht in eine kürzere Laufzeit. Die naive Variante profitiert zudem von einer L1-Hit-Rate von 50.44 %, die in der Shared-Memory-Variante auf 23.32 % fällt, da die Zwischenpufferung in Shared Memory den L1-Pfad umgeht.

Der vorquantisierte Bucket-Richtungspuffer (Tabelle 14) senkt die naive Kernel-Laufzeit gegenüber dem Float-Puffer von 0.633 ms auf 0.576 ms, da pro Thread die Umrechnung von Radiant in Grad entfällt und statt eines `float` nur ein `uint8_t` gelesen wird.

Bemerkenswert ist, dass keine der herangezogenen Referenzarbeiten einen quantifizierten Shared-Memory-gegen-naiv-Vergleich speziell für die Non-Maximum Suppression liefert. Für diese Pipeline-Stufe zahlt sich Shared Memory Tiling, anders als beim Gaussian Blur, nicht aus.

Die Pinned-Memory-Variante lässt den Kernel unverändert und wirkt sich, wie beim Gaussian Blur, primär auf den Device-to-Host-Transfer aus, der von 1.925 ms auf 1.640 ms sinkt.

Die vollständigen Messdaten sind in Anhang C (S. 29) aufgeführt.

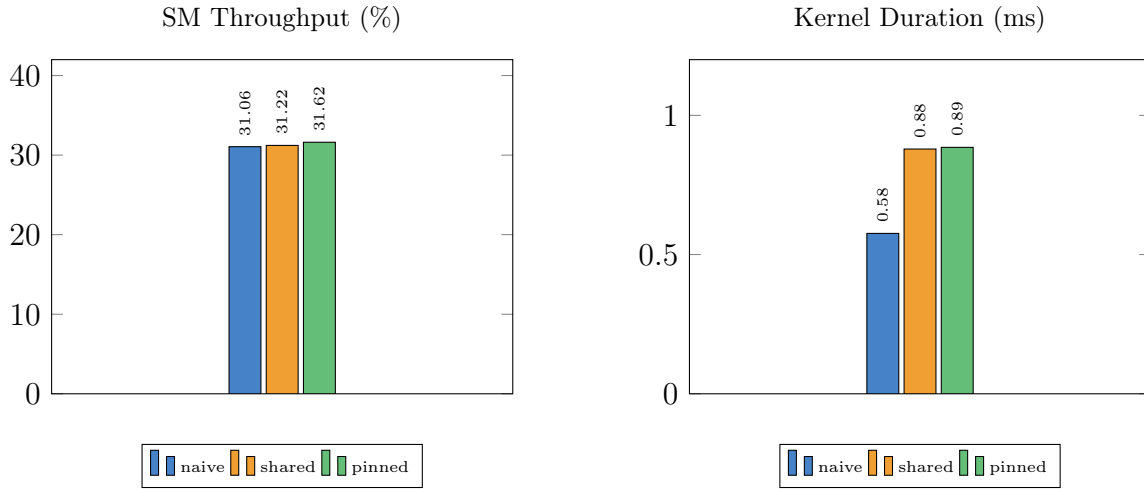


Abbildung 7: NMS-Profilung, SOBEL_OPT=4 (Bucket-Richtung). SM Throughput aus NCU (10 Runs), Kernel Duration aus cudaEvent-Messung (200 Runs Avg).

4.4 Schritt 4: Hysteresis

4.4.1 Performance

Die folgenden Laufzeitwerte entsprechen der TJ-Spalte aus Anhang D und sind Mittelwerte aus 200 Läufen. Der Warmup-Lauf wird nicht in den Vergleich einbezogen.

Variante	H→D [ms]	Kernel [ms]	D→H [ms]	Gesamt [ms]
naive	0.593	0.097	0.657	1.347
shared	0.587	0.157	0.646	1.390
optimiert	0.587	0.081	0.663	1.330

Tabelle 5: Hysteresis-Laufzeiten der drei Varianten, TJ-Messung; Mittelwert aus 200 Läufen.

Die optimierte Variante erzielt mit 0.081 ms die kürzeste Kernelzeit. Gegenüber der naiven Variante entspricht dies einer Reduktion um rund 16.5 %. Im Kernel werden für innere Pixel die acht Nachbarschaftszugriffe vollständig entrollt. Für Randpixel bleibt ein separater, bounds-geprüfter Pfad erhalten. Dadurch werden Schleifen- und Indexierungsaufwand im häufigeren inneren Pfad reduziert, ohne ungültige Speicherzugriffe an den Bildrändern zu erzeugen.

Die Shared-Memory-Variante ist mit 0.157 ms dagegen rund 61.9 % langsamer als die naive Kernelvariante. Zwar lädt jeder Block die Bildkachel einschließlich eines ein Pixel breiten Halos nur einmal aus dem globalen Speicher. Vor der Klassifikation ist jedoch eine `__syncthreads()`-Barriere erforderlich, und die Kachel muss zunächst durch zusätzliche Rand- und Eckzugriffe aufgebaut werden. Für die hier verwendete lokale 8-Nachbarschaft reicht der daraus entstehende Vorteil nicht aus, um diesen Overhead auszugleichen.

Bei der Gesamtzeit fällt der Unterschied zwischen den Varianten deutlich kleiner aus, weil die Transfers den größten Anteil einnehmen. Bei der naiven Variante entfallen 1.250 ms der insgesamt 1.347 ms auf $H \rightarrow D$ und $D \rightarrow H$.

4.4.2 Kernelprofiling

Die Profiling-Tabelle in Anhang D zeigt für die Shared Memory Variante eine deutlich höhere Occupancy, aber niedrigere Cache-Hitrates und eine höhere Kernelzeit. Konkret senkt die Shared Memory Variante den Memory Throughput von 42.91 % auf 36.04 % sowie den DRAM Throughput auf 9.51 % und erhöht die L2-Hit-Rate auf 79.50 %. Diese Werte zeigen, dass die Kachel tatsächlich einen Teil der globalen Speicherzugriffe ersetzt. Gleichzeitig fällt die L1/TEX-Hit-Rate auf 26.46 %, und die Laufzeit steigt. Die Speicherentlastung allein reicht somit nicht aus, um den Synchronisations- und Ladeaufwand zu kompensieren.

Die Einordnung bestätigt die Vermutung, dass das Tiling und die Synchronisation bei dieser kleinen Nachbarschaft mehr Aufwand erzeugen als Nutzen bringen (ähnlich wie im Fall des Sobel-Kernels).

Metrik	naive	shared	optimiert
Compute (SM) Throughput [%]	33.57	41.65	33.60
Memory Throughput [%]	42.91	36.04	33.96
DRAM Throughput [%]	16.50	9.51	19.44
Achieved Occupancy [%]	70.67	90.22	70.28
Warp Cycles per Issued Instruction	25.52	25.77	27.73
L1/TEX Hit Rate [%]	55.28	26.46	51.74
L2 Hit Rate [%]	75.27	79.50	70.54

Tabelle 6: Zusammengefasstes Hysteresis-Kernelprofiling aus Anhang D; zehn Runs je Variante, Warm-up ausgeschlossen.

Die optimierte Variante besitzt nahezu dieselbe Occupancy wie die naive Variante, erreicht aber den niedrigsten Memory Throughput von 33.96 % und den höchsten DRAM Throughput von 19.44 %. Ihre Warp-Zyklen liegen mit 27.73 über den beiden anderen Varianten. Die Profiling-Werte zeigen damit kein einheitliches Bild einer höheren Speicherauslastung. Zusammen mit der niedrigsten gemessenen Kernelzeit zeigt die Laufzeitmessung jedoch, dass die unrolled Global-Memory-Zugriffe auf der verwendeten GPU günstiger sind als das Shared-Memory-Tiling. Insgesamt ist der deutlich niedrigere Memory Throughput interessant und scheint darauf hinzuweisen, dass die unrolled Variante auch einen Einfluss darauf hat, wie aus den verschiedenen Speichern geladen wird (allerdings nicht bei der Sobel-Variante).

4.5 Fused Pipeline

4.5.1 Konzept

In der bisher betrachteten Konfiguration wird nach jeder Pipeline-Stufe das Zwischenergebnis vom Device zurück auf den Host kopiert und im nachfolgenden Schritt erneut

auf das Device übertragen, obwohl die Daten dort direkt weiterverarbeitet werden könnten. Die Fused Pipeline (aktivierbar über `FUSED_PIPELINE=1`, siehe Abschnitt 1.4) verzichtet auf diese redundanten $H \rightarrow D/D \rightarrow H$ -Transfers zwischen den vier Stufen: Das Eingabebild wird einmalig auf das Device kopiert, alle vier Kernel (Gaussian Blur, Sobel, Non-Maximum Suppression, Hysteresis) laufen anschließend nacheinander auf denselben Device-Buffern, und lediglich das finale Ergebnis wird am Ende einmalig zurück auf den Host kopiert.

4.5.2 Performance

Abbildung 8 vergleicht die Summe der isoliert gemessenen Einzelschritte (jeweils mit eigenem $H \rightarrow D/D \rightarrow H$ -Transfer) mit der Fused Pipeline, die nur einmalig zu Beginn und am Ende transferiert.

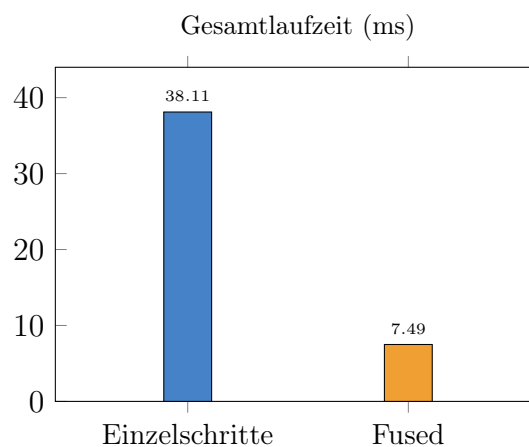


Abbildung 8: Fused Pipeline vs. Summe der isolierten Einzelschritte (MS-Messung).

Durch den Wegfall der redundanten Zwischentransfers sinkt die Gesamtlaufzeit um 30.615 ms bzw. 80.3 %. Da jede Stufe zuvor ihr Ergebnis erst auf den Host und im nächsten Schritt wieder auf das Device kopierte, macht dieser Overhead den Großteil der isolierten Gesamtzeit aus, während die eigentliche Kernel-Rechenzeit vergleichsweise gering bleibt.

5 Fazit

5.1 Fazit

Über alle vier Pipeline-Stufen zeigt sich ein klares Muster: Shared Memory Tiling lohnt sich nur dort, wo redundante Global-Memory-Zugriffe stark ins Gewicht fallen, etwa beim Gaussian Blur mit großem Radius (Abschnitt 4.1). Bei den kleinen Nachbarschaften von Sobel, NMS und Hysteresis überwiegt dagegen der Synchronisations-Overhead, sodass die naive bzw. unrolled Variante schneller bleibt. Pinned Memory verbessert durchgängig den D→H-Transfer, ohne den Kernel zu verändern. Den größten Effekt erzielt die Fused Pipeline (Abschnitt 4.5), die durch den Wegfall der Zwischentransfers die Gesamtlaufzeit um über 80 % reduziert.

Aus der Sicht von Matthias Stefan zeigte dieses Praktikum insbesondere die Herausforderungen sauberen Performance-Messens in der Praxis auf: Ohne kontrollierte Laborbedingungen, etwa hinsichtlich konstanter GPU-Temperatur oder isolierter Hintergrundprozesse (vgl. Abschnitt 1.5), unterliegen Messwerte einer gewissen Schwankungsbreite. Daraus ergibt sich eine grundsätzlichere methodische Frage: Inwieweit sind unter Laborbedingungen gewonnene Messwerte aussagekräftig, wenn die Zielumgebung, insbesondere im Consumer-Bereich, diese Bedingungen ohnehin nie erfüllt? Realitätsnahe, durch Hintergrundlast beeinflusste Messbedingungen könnten den späteren Einsatzkontext unter Umständen treffender abbilden als ein steriles Setup. Die im Praktikum gewonnenen Erkenntnisse zu Speicherzugriffsmustern und Transfer-Overhead sind zudem auf angrenzende Bereiche wie 3D-Rendering und Vulkan übertragbar.

Aus der Sicht von Thomas Jurczyk «<TODO>»

Als Ausblick bleibt festzuhalten, dass sich alle Messungen auf ein einzelnes, wenn auch großes Bild (5120×2880) beschränken. Effekte wie Speicherbandbreitenlimitierung oder Occupancy-Grenzen dürften bei deutlich größeren oder kleineren Bildgrößen anders bzw. stärker ausfallen, sodass eine systematische Analyse über mehrere Bildgrößen hinweg ein sinnvoller nächster Schritt wäre.

Literatur

- [1] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, November 1986.
- [2] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson Education, New York, New York, fourth, global edition edition, 2018.
- [3] Matthew Horvath, Michael Bowers, and Shadi Alawneh. Canny edge detection on gpu using cuda. In *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0419–0425, 2023.
- [4] Kohei Ogawa, Yasuaki Ito, and Koji Nakano. Efficient canny edge detection using a GPU. In *2010 First International Conference on Networking and Computing*, pages 279–280, 2010.
- [5] Yupu Song, Cailin Li, Shiyang Xiao, Qinglei Zhou, and Han Xiao. A parallel canny edge detection algorithm based on OpenCL acceleration. *PLoS ONE*, 19(1):e0292345, January 2024.
- [6] Rishita Yadav and Harshul Gupta. Optimizing CUDA kernels for high-performance canny edge detection. *International Journal of Science, Engineering and Technology*, 13(4), 2025.

Anhang

A Performance-Rohdaten Gaussian Blur

Metrik	naive	shared	pinned
<i>Warm-up Run</i>			
H→D [ms]	3.062	5.660	2.709
Kernel [ms]	2.606	1.697	1.688
D→H [ms]	7.631	7.020	2.502
Total [ms]	13.299	14.377	6.899
<i>200 Runs Avg</i>			
H→D [ms]	3.548	3.475	3.135
Kernel [ms]	1.669	0.990	0.987
D→H [ms]	3.068	3.001	1.740
Total [ms]	8.286	7.466	5.862

Tabelle 7: cudaEvent-Messungen, BLUR_RADIUS = 2

Metrik (NCU, 10 Runs Avg)	naive	shared	pinned
Compute (SM) Throughput [%]	68.31	66.25	66.54
Memory Throughput [%]	61.10	66.25	66.54
DRAM Throughput [%]	5.81	8.42	7.60
Achieved Occupancy [%]	76.50	92.67	92.65
Warp Cycles Per Issued Instruction	8.79	16.46	16.33
L1/TEX Hit Rate [%]	92.94	27.32	27.29
L2 Hit Rate [%]	87.75	88.17	89.12

Tabelle 8: NCU-Profilng, BLUR_RADIUS = 2

Metrik	naive	shared	pinned
<i>Warm-up Run</i>			
H→D [ms]	2.589	1.757	1.781
Kernel [ms]	35.700	12.575	13.182
D→H [ms]	6.035	6.183	1.836
Total [ms]	44.324	20.515	16.799
<i>200 Runs Avg</i>			
H→D [ms]	2.512	2.375	2.057
Kernel [ms]	22.669	7.537	7.325
D→H [ms]	2.142	2.036	1.366
Total [ms]	27.308	11.948	10.747

Tabelle 9: cudaEvent-Messungen, BLUR_RADIUS = 9

Metrik (NCU, 10 Runs Avg)	naive	shared	pinned
Compute (SM) Throughput [%]	64.66	91.50	91.46
Memory Throughput [%]	64.66	91.50	91.46
DRAM Throughput [%]	1.06	1.69	1.79
Achieved Occupancy [%]	95.52	94.84	94.83
Warp Cycles Per Issued Instruction	12.56	13.78	13.77
L1/TEX Hit Rate [%]	99.27	45.55	45.54
L2 Hit Rate [%]	63.36	82.96	87.35

Tabelle 10: NCU-Profilung, BLUR_RADIUS = 9

B Sobel Messungen

Metrik	naive		shared		split		pinned		bucket	
	MS	TJ	MS	TJ	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>										
H→D [ms]	1.835	0.525	3.372	0.525	0.000	0.531	1.293	0.614	1.400	0.528
Kernel [ms]	0.717	0.179	1.062	0.246	0.000	0.310	0.539	0.238	0.466	0.109
D→H [ms]	32.075	14.472	30.810	16.103	0.000	7.304	7.794	2.619	11.478	7.378
Total [ms]	34.626	15.176	35.244	16.874	0.000	8.146	9.626	3.471	13.344	8.015
<i>200 Runs Avg</i>										
H→D [ms]	1.601	0.528	1.715	0.527	0.000	0.533	1.596	0.533	1.926	0.528
Kernel [ms]	0.567	0.140	1.000	0.211	0.000	0.281	0.569	0.145	0.453	0.079
D→H [ms]	27.348	9.649	27.210	9.747	0.000	1.409	7.592	2.574	3.720	4.271
Total [ms]	29.516	10.317	29.925	10.484	0.000	2.223	9.757	3.253	6.099	4.878

Tabelle 11: Sobel-Filter, cudaEvent-Messungen. MS: Matthias Stefan, TJ: Thomas Jurczyk.

Metrik (NCU, Avg)	naive	shared	optimized	bucket	split
Compute (SM) Throughput [%]	61.70	62.36	62.24	57.07	45.46
Memory Throughput [%]	53.36	37.52	54.02	57.13	56.23
DRAM Throughput [%]	53.36	34.06	54.02	28.73	44.00
Achieved Occupancy [%]	81.12	90.04	81.03	78.43	76.65
Warp Cycles Per Issued Instruction	15.39	17.10	15.22	20.13	26.12
L1/TEX Hit Rate [%]	62.69	12.79	62.68	67.47	43.77
L2 Hit Rate [%]	68.43	39.65	68.73	82.03	65.26

Tabelle 12: NCU-Profilng der Sobel-Implementierungen

C NMS Messungen

Metrik	naive		shared		pinned	
	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>						
H→D [ms]	10.035	3.130	14.527	3.481	6.178	2.854
Kernel [ms]	0.603	0.161	1.089	0.278	1.002	0.283
D→H [ms]	6.540	2.476	7.319	3.222	1.315	0.531
Total [ms]	17.178	5.767	22.935	6.981	8.496	3.669
<i>200 Runs Avg</i>						
H→D [ms]	9.865	2.973	13.759	2.907	6.379	2.598
Kernel [ms]	0.633	0.192	1.015	0.216	1.009	0.226
D→H [ms]	1.925	0.730	2.678	0.693	1.361	0.524
Total [ms]	12.423	3.895	17.453	3.815	8.749	3.348

Tabelle 13: NMS mit Float-Richtungspuffer (SOBEL_OPT=2).
cudaEvent-Messungen. MS: Matthias Stefan, TJ: Thomas Jurczyk.

Metrik	naive		shared		pinned	
	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>						
H→D [ms]	3.778	1.295	5.521	1.288	2.484	1.202
Kernel [ms]	0.571	0.095	1.037	0.147	0.881	0.271
D→H [ms]	6.416	3.049	9.325	2.871	1.455	0.534
Total [ms]	10.765	4.439	15.883	4.306	4.819	2.007
<i>200 Runs Avg</i>						
H→D [ms]	4.020	1.150	7.221	1.149	3.048	1.050
Kernel [ms]	0.576	0.096	0.879	0.141	0.885	0.140
D→H [ms]	1.925	0.697	3.525	0.695	1.640	0.522
Total [ms]	6.521	1.942	11.625	1.984	5.572	1.712

Tabelle 14: NMS mit vorquantisiertem Bucket-Richtungspuffer (SOBEL_OPT=4). cudaEvent-Messungen.

MS: Matthias Stefan, TJ: Thomas Jurczyk.

Metrik (NCU, 10 Runs Avg)	naive	shared	pinned
Compute (SM) Throughput [%]	31.06	31.22	31.62
Memory Throughput [%]	26.93	24.65	24.96
DRAM Throughput [%]	26.15	17.43	18.20
Achieved Occupancy [%]	85.10	93.37	90.21
Warp Cycles Per Issued Instruction	31.17	22.51	23.06
L1/TEX Hit Rate [%]	50.44	23.32	23.23
L2 Hit Rate [%]	85.18	85.64	85.81

Tabelle 15: NCU-Profilng der NMS-Kernel, SOBEL_OPT = 4 (Bucket-Richtung). 10 Runs, Warm-up ausgeschlossen.

D Hysteresis Messungen

Metrik	naive		shared		optimized	
	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>						
H→D [ms]	2.954	0.926	1.844	0.860	1.872	0.922
Kernel [ms]	0.477	0.090	0.794	0.152	0.421	0.071
D→H [ms]	5.056	3.408	5.069	3.439	7.582	3.517
Total [ms]	8.487	4.425	7.707	4.451	9.875	4.511
<i>200 Runs Avg</i>						
H→D [ms]	2.093	0.593	1.643	0.587	1.651	0.587
Kernel [ms]	0.481	0.097	0.783	0.157	0.401	0.081
D→H [ms]	1.901	0.657	1.739	0.646	1.771	0.663
Total [ms]	4.475	1.347	4.164	1.390	3.823	1.330

Tabelle 16: Hysteresis Kernel. cudaEvent-Messungen.
MS: Matthias Stefan, TJ: Thomas Jurczyk.

Metrik (NCU, 10 Runs Avg)	naive	shared	opt
Compute (SM) Throughput [%]	33.57	41.65	33.60
Memory Throughput [%]	42.91	36.04	33.96
DRAM Throughput [%]	16.50	9.51	19.44
Achieved Occupancy [%]	70.67	90.22	70.28
Warp Cycles Per Issued Instruction	25.52	25.77	27.73
L1/TEX Hit Rate [%]	55.28	26.46	51.74
L2 Hit Rate [%]	75.27	79.50	70.54

Tabelle 17: NCU-Profilng der Hysteresis-Implementierungen